

Project 2: Music Catalogue

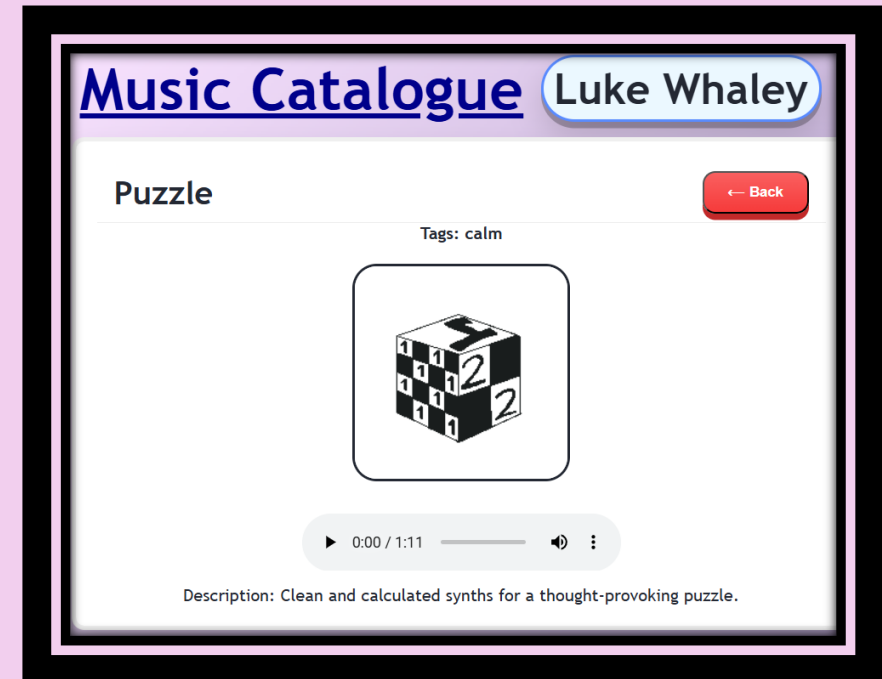
Luke Whaley



GitHub Link: <https://lukewhaley.github.io/Sophomore-Project-2-Music-Catalogue>

What is my Project?

My project is a music catalogue that showcases some of my own music. This website allows people to easily access and listen to my songs by using a tag system to sort by different styles of music.



Main View

In the main view, you can see a list of individual songs. By clicking on one of these songs, you will be taken to the song view.

You can also search for specific styles of music by using the tag button at the top of the page. Multiple tags can be selected at once, which allows the user to filter out any songs that do not contain every active tag.

Music Catalogue

Luke
Whaley

Tags:

ambient

upbeat

sad

funny

calm

intense

3D Shack

Wave

Blooper Battle

Pizza Time!

Extreme

Puzzle

Ginza Line - 1:00 AM

Song View

In the song view, you can listen to the song you have selected by pressing play.

The song's title and tags are displayed at the top of the screen. Additionally, a corresponding image and a brief description for the song are included on this page.

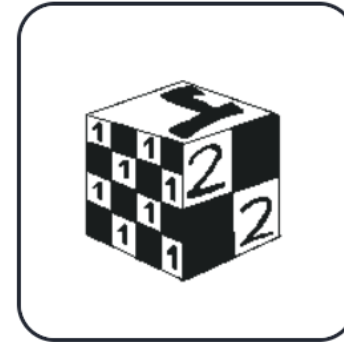
Music Catalogue

Luke Whaley

Puzzle

← Back

Tags: calm



▶ 0:00 / 1:11 ———— 🔊 ⋮

Description: Clean and calculated synths for a thought-provoking puzzle.

Functions used in my Project:

My project uses 5 different functions:

- openSong(key)
- backHome()
- stopAudio(id)
- setTag(tag)
- filterSongs()

Function 1: openSong(key)

This function sends the user to the song view by setting the display of the home and song styles. This function is run when a song is clicked in the main view.

This function also updates the title, image, audio file, and description within the song view depending on which song the user clicked on. This is determined by the 'key' parameter, which is set to the data attribute 'data-song' that is present in each cover card. This parameter is then used to find each individual song's title, tags, and description, which are contained within 'const SONGS.'

The corresponding image and mp3 file of each song are both named directly after the 'data-song' attributes and are contained within the "images" and "music" files.

```
function openSong(key) {  
  const d = SONGS[key];  
  if (!d) return;  
  songTitle.textContent = d.title;  
  songTags.textContent = "Tags: " + d.tags;  
  
  // update image  
  const imgEl = document.getElementById("songImage");  
  imgEl.src = `images/${key}.png`;  
  
  // update audio  
  const audioEl = document.getElementById("inPage");  
  audioEl.src = `music/${key}.mp3`;  
  
  songDescription.textContent = "Description: " + d.description;  
  
  home.style.display = 'none';  
  song.style.display = 'block';  
}
```

```
<div class="covers">  
  <figure class="cover-card" data-song="shack3D">  
    <figcaption class="title">3D Shack</figcaption>  
  </figure>  
</div>
```

```
const SONGS = {  
  shack3D: { title: "3D Shack", tags: "calm", description: "So retro." },  
  
  document.querySelectorAll('.cover-card').forEach(card => {  
    card.addEventListener('click', () => openSong(card.dataset.song));  
  });
```

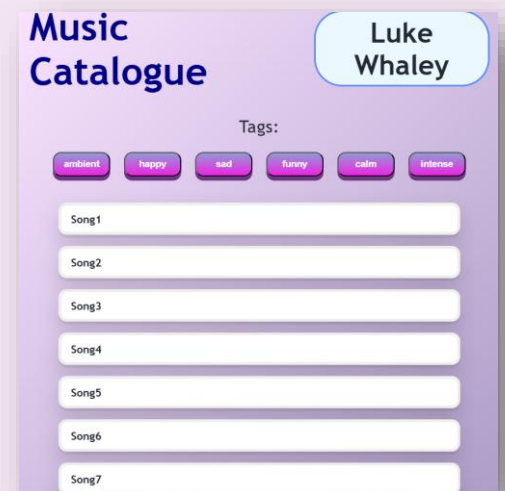
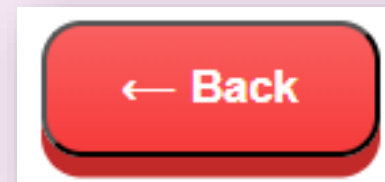

Function 2: backHome()

This function returns to the main view of the site by setting the display of the song style and home style.

This function is used when the back button is pressed from the song view.

This function also calls the stopAudio(id) function, which stops any song that is still playing within the song view.

```
function backHome() {  
    song.style.display = 'none';  
    home.style.display = 'flex';  
    stopAudio("musicViewSong");  
}
```

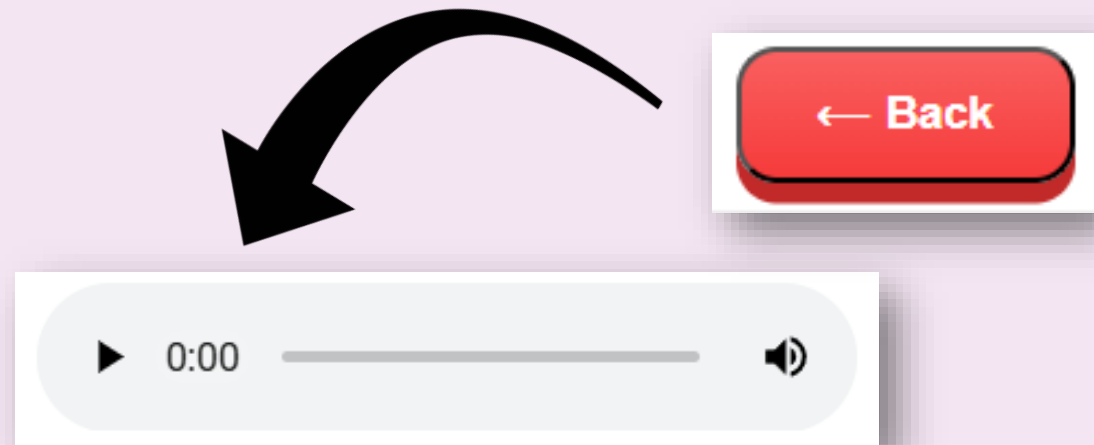


Function 3: stopAudio(id)

This function stops the audio from playing in the song view when you press the back button and resets the time to 0:00. The 'id' parameter is used to find the specific audio player that the function will stop from playing.

Without this function, the song would continue to play even after returning to the main view. The only way for the user to stop the song at this point would be to either return to the song view and press the pause button or wait for the song to end.

```
function stopAudio(id) {  
  document.getElementById(id).pause();  
  document.getElementById(id).currentTime = 0;  
}
```



Function 4: setTag(tag)

This function sets which search tags are currently being used when a tag button is pressed. The function checks if the tag that was clicked is already active:

- if the tag is not active, the tag is added to the set of active tags, and the button is set to a pressed visual style.
- if the tag is already active, the tag is removed from the set of active tags, and the button is set to an unpressed visual style.

Finally, the function `filterSongs()` is run to change which songs are currently visible in the main view.

```
function setTag(tag) {  
  const btn = [...document.querySelectorAll('.btn2')].find(b => b.textContent === tag);  
  if (!btn) return;  
  
  // toggle active state  
  if (activeTags.has(tag)) {  
    activeTags.delete(tag);  
    btn.classList.remove('active');  
  } else {  
    activeTags.add(tag);  
    btn.classList.add('active');  
  }  
  
  filterSongs();  
}
```



Unactive tag:



Active tag:

Function 5: filterSongs()

This function filters out the songs in the main view based on the active tags by changing their style display.

This function goes through every song card to check if they should be visible or not.

If no tags are active, all songs will be visible. If there are active tags, the function checks if every active tag is included within each song.

- If an active tag is not included, the song's display is set to 'none.'
- If all active tags are included, the song will be set to visible.

```
function filterSongs() {  
  document.querySelectorAll('.cover-card').forEach(card => {  
    const songKey = card.dataset.song;  
    const song = SONGS[songKey];  
  
    // If no tags are active, show all songs  
    if (activeTags.size === 0) {  
      card.style.display = '';  
      return;  
    }  
  
    // Otherwise show only those matching one or more active tags  
    const songTags = song.tags.split(',').map(t => t.trim());  
    const hasAllTags = [...activeTags].every(tag => songTags.includes(tag));  
  
    card.style.display = hasAllTags ? '' : 'none';  
  });  
}
```